LNH-MusicTag 开发文档

版本：v1.4.9 · 仓库：github.com/lahuolou/lnh-musictag · 协议：MIT
本文面向开发者：介绍项目架构、模块职责、数据模型、API、插件扩展点、音频识别与一键刮削流水线、封面优化、发布流程。

目录

1. 项目概述
2. 系统架构
3. 目录结构
4. 数据模型
5. 音频内容识别流水线
6. 一键刮削流水线
7. 批量刮削字段模型与封面优化
8. 后端模块详解
9. 插件框架（核心扩展点）
10. 前端详解（Vue3）
11. API 参考
12. 部署
13. 发布流程（CI/CD）
14. 更新检查机制
15. 常见问题排查
16. 测试
17. 安全说明

1. 项目概述

LNH-MusicTag 是一个**自托管音乐库标签管理工具**：扫描本地音乐目录，读写音频标签（标题/艺术家/专辑/封面/歌词等），多源刮削补全元数据，智能去重，并支持格式转换、乱码修复、简繁体转换、音频内容识别等功能。只提供 Web 后台（账号/密码/API Key 均存本地数据库）。

核心设计目标：

- **单二进制交付**：Go 后端通过 `go:embed` 内嵌 Vue3 构建产物，运行只有一个可执行文件。
- **无 CGo**：标签读写采用 go-taglib（WASM 封装 TagLib），SQLite 采用 modernc.org/sqlite（纯 Go），全静态编译。
- **插件化扩展**：刮削源 / 工具 / 下载器 / 服务协议统一走插件注册表；第三方插件以 zip 导入，自带 Web 面板，新能力无需改宿主代码。

- **后台化任务**：扫描、批量刮削、识别、转换、修复等全部异步执行，进度可视、成功/失败/跳过统计、可取消、刷新/换设备可恢复。
- **一条流水线**：识别、一键刮削、乱码修复、简繁转换共用同一套异步任务框架与公共实现，避免重复代码。

1.1 技术栈

层	技术	说明
后端	Go 1.27（标准库 net/http + ServeMux 路由）	无第三方 Web 框架
标签读写	github.com/sentriz/go-taglib（WASM）	免 CGo，支持多值标签
音频指纹	chromaprint（fpcalc 子进程）+ AcoustID API	音频内容识别
元数据补全	MusicBrainz Web Service v2	Recording ID → 完整元数据
简繁转换	OpenCC（zhscript 封装）	简繁互转
存储	modernc.org/sqlite（纯 Go SQLite）	配置 + 曲目统一入库
前端	Vue 3 + Vite	无 UI 框架，自绘样式
部署	Docker 多阶段构建 → ghcr.io 镜像	CI 自动构建推送
更新检查	GitHub API + jsDelivr CDN + raw 文件	三源容错

1.2 版本规划

- 版本号语义化递增：v1.4.1 → v1.4.2 → … → v1.4.50 → **v1.5.0** → v1.5.1 …
- 每次发布必须：递增 `main.go` 的 `AppVersion`、更新根目录 `VERSION` 文件、创建对应 GitHub Release tag（tag 名带 `v` 前缀）。
- 更新检查依据 `VERSION` 文件与 Release tag，语义化逐段比较（`1.4.50 → 1.5.0` 正确判新）。
- **版本不同步递增会导致项目检测不到更新**——这是登录后"检查更新"误报"已是最新"的最常见原因。

2. 系统架构

```
浏览器 (Vue3 SPA)
   │  HTTP /api/*  (Cookie 会话 + CSRF + 限流)
   ▼
Go 后端 (单二进制，端口 10248)
   ├── 会话鉴权 auth.go          ── 登录/登出/改密/会话 Cookie
   ├── 安全防护 security.go      ── CSRF / XSS / SQL 注入防护 / 输入净化
   ├── 插件框架 plugin.go        ── 内置插件注册表 + 第三方插件（zip 导入/面板/JSON 协议）
   ├── 刮削引擎 internal/scrape  ── 各源实现 + MultiSource 调度（语言优先）+ 封面下载加固
   ├── 标签读写 internal/taglibx ── go-taglib 封装
   ├── 任务引擎 scrapejob.go     ── 批量任务（进度/恢复/结果统计/取消）+ 专辑级封面缓存
   ├── 工具链 convert.go / fixencoding.go / zhscript.go / identify.go / renameclean.go
   ├── 流水线 pipeline.go        ── 一键刮削（可选去重 → 修复 → 转换 → 刮削）
   ├── 曲目存储 tracks_db.go     ── SQLite 持久化（重启不丢、增量落库）
   ├── 操作日志 activity.go      ── 记录刮削/转换/修复/识别/插件启停等动作
   ├── 更新检查 version.go       ── 多源最新版本查询 + 三态结果
   └── 静态资源 static.go        ── index.html no-cache / 哈希资源 immutable
         │
         ├── SQLite (lnh.db)   ── settings 表（账号/Key/选项/插件配置）
         │                        tracks 表（曲目 JSON）
         └── 音乐目录 (/music) ── 读写标签/封面/歌词文件
```

2.1 请求链路

1. 公开首页 `/`：无需登录（品牌 + 介绍 + 搜索占位），登录入口弹层。
2. 登录后进入后台：顶栏（曲目数/列表/首页/齿轮菜单），合并页布局（左侧列表 + 右侧编辑/批量/去重 + 第三方插件 tab）。
3. 所有 `/api/*` 除 `login` 外均要求会话 Cookie，经 `csrfGuard` + `limitBody` 中间件。
4. 前端 `api.js` 统一封装 fetch，自动带 `X-Requested-With: LNH-MusicTag` 头（CSRF 校验需要）。

3. 目录结构

```
项目根
├── main.go              程序入口：路由注册、中间件、静态资源、服务启动、store
├── auth.go              会话与账号：登录/登出/改密/随机初始密码/会话 Cookie
├── security.go          安全：CSRF、XSS 净化、SQL 注入防护、敏感键校验
├── plugin.go            插件框架：内置注册表/快照/启停/配置持久化
├── plugins_api.go       插件注册与 API：刮削源注册、GET/POST /api/plugins
├── third_party.go       第三方插件：zip 导入、plugin.json 声明、fs 扫描接口、panel 静态服务
├── version.go           更新检查：多源查询、三态结果、语义化版本比较
├── static.go            静态缓存策略：index.html no-cache、assets immutable
├── scrapejob.go         批量任务引擎：进度、恢复、统计、取消、专辑级封面缓存、封面失败上报
├── pipeline.go          一键刮削流水线：自动去重(可选)→乱码修复→繁转简→补艺术家→刮削
├── identify.go          音频内容识别：格式分流→转码→指纹→AcoustID→MB→写标签→重命名
├── fixencoding.go       乱码修复（GBK/UTF-8 错乱检测与重写；与流水线共用 repairTrackText）
├── zhscript.go          简繁体转换（OpenCC；识别/修复结果统一转简体）
├── convert.go           格式转换（ffmpeg 转码）
├── ffmpeg.go            FFmpeg 插件化：路径探测、状态、一键安装、卸载
├── renameclean.go       修正文件后缀（探测真实格式后重命名，如 .mp3.flac → .flac）
├── activity.go          操作日志（记录动作，可删除/清空）
├── helpers.go           通用工具：writeJSON、路径安全、标签标准化等
├── tracks_db.go         曲目 SQLite 持久化：落库/加载/外部删除清理
├── *.go *_test.go       测试
├── internal/
│   ├── store/config.go  SQLite 键值配置存储（settings 表）
│   ├── model/model.go   数据模型：Track、DuplicateGroup、音频扩展名
│   ├── taglibx/         go-taglib 标签读写封装
│   ├── dedup/           去重策略（哈希/格式/标签）
│   ├── finger/          Chromaprint 指纹封装（含 AcoustID 用压缩指纹）
│   ├── audiofmt/        音频格式探测
│   └── scrape/          刮削源：sources.go（封面下载加固）+ 各源实现（含 MusicBrainz 客户端）
├── web/
│   ├── package.json / vite 配置
│   └── src/
│       ├── main.js      入口
│       ├── App.vue      根组件：首页/后台切换、顶栏、齿轮菜单、全局弹窗、第三方插件 tab
│       ├── store.js     全局状态（reactive）+ 任务轮询 + 更新检查
│       ├── api.js       fetch 封装
│       ├── i18n.js      中英双语字典
│       ├── style.css    全局样式（明暗主题变量 + 移动端适配）
│       ├── views/       Home（公开首页）/ Login / Settings / Plugin
│       └── components/  MusicTree（左列表）/ EditPanel / BatchPanel /
│                         DedupPanel / ExtPanel（第三方插件 iframe 面板）/ Toast 等
├── Dockerfile           多阶段：Vue 构建 → Go 静态编译 → alpine 运行
├── docker-compose.yml   推荐部署（端口/卷/环境变量/FFmpeg 开关）
├── entrypoint.sh        LNH_INSTALL_FFMPEG=true 时启动前自动安装 ffmpeg
├── VERSION              当前版本号（更新检查用）
└── README.md            使用说明（中英双语）
```

4. 数据模型

4.1 settings 表（SQLite 键值）

键	说明
`admin_user` / `admin_pass`	后台账号 / 密码（bcrypt 哈希）
`session_secret`	会话签名密钥（首启生成）
`ffmpeg_path`	旧键：FFmpeg 路径（兼容迁移）
`plugins`	插件启停与配置 JSON（全量持久化，含第三方插件）
`sources_enabled`	旧键：刮削源开关（兼容迁移）

各 API Key（Last.fm / Spotify / Discogs / Jamendo / AcoustID 等）**迁移到插件配置**中：Key 由对应插件持有（`settings.plugins` 内），设置页不再暴露 Key 输入。

4.2 tracks 表

列	说明
`id`	稳定 ID = SHA256[:16]
`path`	服务器绝对路径
`data`	Track 完整 JSON
`seq`	扫描顺序（重启后按序恢复，保持"先扫先排"）

启动时加载全部曲目（文件已外部删除的自动丢弃），扫描/编辑/重命名/删除均同步落库。

4.3 Track 结构（model）

```
type Track struct {
    ID, Path, FileName, Ext string
    Size int64
    SHA256 string          // 去重策略1：内容哈希
    Duration float64        // 秒
    Bitrate, SampleRate, Channels uint
    HasCover, HasLrcFile bool
    HasTrad bool            // 标签含繁体（OpenCC 检测，供"繁体"筛选）
    Garbled bool            // 标签疑似乱码（GBK/UTF-8 错乱）
    NeedsIdentify bool      // 无文件名与标签信息 → 需音频内容识别
    Tags map[string]string  // 扁平化标签（标准键）
}
```

5. 音频内容识别流水线（identify.go）

按"扫描 → 分流 → 指纹 → AcoustID → MusicBrainz → 写标签 → 重命名"流水线实现，对标 AcoustID + MusicBrainz 官方方案：

```
音频文件
   │  14 种格式：.mp3 .m4a .aac .flac .alac .wav .aiff .ogg
   │              .opus .wma .ape .wv .dsf .dff（兼容 .mpc/.mka）
   ▼
格式分流（identifyNeedsTranscode）
   ├── 免转码族：.mp3 .flac .wav .ogg .opus → 直接指纹
   └── 其他（含 DSD .dsf/.dff）→ FFmpeg 转 WAV（16bit/44.1kHz/双声道）再指纹
   ▼
指纹生成（internal/finger）
   └── fpcalc -json → Chromaprint 压缩指纹 + 时长（AcoustID 需要压缩指纹而非 raw）
   ▼
AcoustID 查询（3 QPS 限速，acOnce）
   └── /v2/lookup?client=KEY&meta=recordings+releasegroups+compress
       → MusicBrainz Recording ID + 基础元数据（score 置信度）
   ▼
MusicBrainz 补全（1 QPS 限速，mbOnce）
   └── LookupRecording(mbid, artists, releases, release-groups, tags)
       → 标题/艺术家/专辑/年代/流派完整元数据
   ▼
写标签 + 重命名（finalizeIdentify 公共收尾）
   ├── 标题/艺术家/专辑/流派自动转简体（AcoustID 与库内回退两路径都转）
   ├── taglibx 写标签 → 抓歌词 → refreshTrack 落库
   └── 重命名"艺术家 - 标题.后缀"（不覆盖已存在目标）
```

关键行为：

- **未配 AcoustID Key / 未命中**：回退**库内指纹匹配**（指纹余弦相似度阈值 0.85），与已标注曲目比对补全。
- **指纹库异步构建**：`POST /api/identify` 立即返回 `{jobId, total}`；任务首步先构建指纹库（`Current="构建指纹库…"`），随后逐曲目识别。
- **识别判定**：`filenameNoInfo`（文件名空/Unknown/乱码/纯数字如 track01）+ `needsIdentify`（文件名无信息且 TITLE 空或等于去后缀 basename）。
- **格式探测**：`.mp3.flac` 这类多后缀文件先探测真实编码格式再重命名正确后缀。

6. 一键刮削流水线（pipeline.go）

一键刮削把日常整理步骤串成一条流水线，逐曲目 × 阶段执行，**每个（曲目 × 阶段）占一个进度点**（进度按曲目计数）：

```
阶段 0（可选）自动去重 —— 勾选才执行，默认不勾选
   按三种口径合并分组（与去重页一致）：
   ① 艺术家+标题一致（同人同歌不同格式/版本）
   ② 同名多格式（广岛之恋.mp3 + 广岛之恋.flac）
   ③ SHA256 内容一致（完全相同副本）
   每组保留音质最好的一首（无损优先；hash 组保留第一首），
   其余删除文件并从列表移除。任一组的 keep 优先于其他组的删除
   （同一曲目同时出现在多组时不会被误删）。
阶段 1 乱码修复 —— stageFixEncoding，与 /api/fix-encoding 共用 repairTrackText
阶段 2 繁体转简体 —— stageScript，OpenCC t2s
阶段 3 文件名补艺术家 —— stageFixArtist（文件名不含艺术家时补）
阶段 4 刮削元素 —— stageScrape（歌名+海报+歌词；源没搜到海报自动匹配下一条源）
```

- 每阶段从 store 取**最新状态**（前一阶段可能重写标签/重命名/删除）。
- `Stage` / `StageLabel`（"刮削元素（3/4）"）单独显示当前阶段，前端实时更新列表不整表刷新。
- **取消**：`POST /api/jobs/{id}/cancel` → 任务在下一步前停止，状态 `cancelled`。
- 曲目被去重阶段删除后，其余阶段对该曲目直接计完成（done 仍能到 total）。

7. 批量刮削字段模型与封面优化

7.1 字段模型（29 项）

`ScrapeFields` 覆盖 29 个可勾选字段，按常用度排序（海报 → 核心标签 → 操作项 → 扩展标签 → 音频属性；前端展示与后端解析共用 `fieldNames()`）：

- **海报**
- **核心标签**：歌名 / 艺术家 / 专辑 / 专辑艺术家 / 风格 / 年代 / 曲目号 / 歌词（内置方式：抓歌词写内嵌标签，兼容旧 LRC 别名）
- **操作项**：文件名（按「艺术家 - 标题.后缀」重命名）
- **扩展标签**：作曲家 / 作词家 / 专辑类型 / 语言 / 简介（并入 COMMENT）/ 光盘编号 / 总光盘数 / 总音轨数 / 曲目增益 / 专辑增益 / 曲目峰值 / 专辑峰值
- **音频属性**（写标签自定义键，无需源搜索）：比特率 / 采样率 / 位深度 / 声道 / 时长 / 文件大小

前端提供**全选/反选**（批量刮削元素区第一位）与**智能跳过**开关（置顶）。曲目列表页展示与批量刮削元素**相同的全部字段列**（窄屏自动隐藏低优先级列并支持横向滚动）；**一键刮削**与批量刮削使用同一完整字段集。属性与操作项**不依赖源搜索**：勾选后即使搜索无结果也会执行（属性写本地探测值、LRC 写盘、文件名重命名）。

`fieldsFromList()` 支持全部中文别名（歌名/标题、流派/风格、年代/年份、海报/封面、简介/描述、歌词(LRC文件)/lrc 等）。

7.2 智能跳过

`applySkipFilled`：乱码文本与 "Unknown/未知" 占位视为"未填充"（`tagNeedsFill`），刮削阶段会修正而不是跳过；LRC 已存在、文件名已规范则跳过对应操作。

7.3 存疑自动识别

批量刮削（`runBatch`）与一键刮削（`stageScrape`）在**配置了 AcoustID Key** 时，对存疑曲目（`trackIsDoubtful`：文件名艺术家为 Unknown/未知/track01、标签艺术家空/乱码/占位、或无有效标题）先执行音频识别：

```
存疑曲目 → fpcalc 指纹 → AcoustID 查询(3QPS) → MusicBrainz Recording 补全(1QPS)
        → 写标签（统一转简体）→ 抓歌词 → 按「艺术家 - 标题.后缀」重命名
        → 识别产生新 ID 后，后续刮削基于新曲目继续
```

未配 Key 或 AcoustID 未命中 → 回退**库内指纹匹配**（相似度 ≥0.85，不消耗在线额度）；仍无结果则继续刮削兜底。消息只报告实际执行的动作。

7.4 封面优化（本轮核心）

封面获取命中率从约 14% 提升到 100%（实测 1691 首库：507 缺 → 0 缺）。三层手段：

① 下载加固（internal/scrape/sources.go）

- 新增 `fetchImage()`：带 Referer（qq/kugou/kuwo/netease/migu 国内源统一 `https://y.qq.com/` 防盗链）、失败重试一次、响应校验（Content-Type 非 text/html、长度 ≥200B）。
- `FetchCover` 全部走 `fetchImage`；`FetchCoverFallback(query, prefer, titleOnly)`：完整 query 全链失败后用**纯歌名**再遍历全部源（iTunes/MusicBrainz 对纯标题命中更高）。

② 专辑级封面缓存（scrapejob.go / pipeline.go）

- 批量（`runBatch`）与一键（`stageScrape`）引入 `coverCache map[string][]byte`（key = 源 AlbumID 或 源+专辑名），**同专辑曲目复用封面**，减少高频请求。
- 写封面成功后 `t.HasCover = true` 同步内存；`scrapeOneTrack` 末尾未重命名时补 `refreshTrack(store, t.Path)` 落库（修复重启后 hasCover 丢失——"海报"筛选统计不准的根因）。
- 封面失败错误串进结果消息（`海报失败: <err>` / `海报写入失败: <err>`），日志可见、可排查。

③ 限流规避结论

批量高频请求是封面失败的主因（单首重放即成功）。优化后批量 480 首 407 命中（84.8%），对剩余失败曲目**自动再跑一轮即 100% 补齐**；配合缓存与重试，后续批量命中率显著提升。

8. 后端模块详解

8.1 入口与路由（main.go）

- `AppVersion` 常量 = 当前版本号（发布时必须递增）。
- 路由：公开 `POST /api/login`、`POST /api/logout`、`GET /api/me`；其余全部挂 `pm`（plugins mux），统一套 `requireAuth → csrfGuard → limitBody`。
- 静态资源：`go:embed` 内嵌 `web/dist`，`/api/*` 优先于静态。

8.2 会话鉴权（auth.go）

- 登录成功 → 生成签名会话 Cookie（HttpOnly + SameSite），密钥 `session_secret` 首启随机生成。
- 首次启动无密码 → 自动生成随机初始密码并打印到容器日志；登录后在「设置」页修改（写库）。
- 调试默认账号 admin/admin（仅测试库，`LNH_CONFIG_DIR` 播种）。

8.3 安全防护（security.go）

威胁	防护
CSRF	自定义头 `X-Requested-With` 校验 + Cookie SameSite
XSS	前端 Vue 默认转义；后端对写入标签做 `<` `>` `&` `"` `'` 净化；第三方面板 iframe 沙箱 + CSP 收紧
SQL 注入	全部参数化查询（database/sql 占位符）
路径穿越	所有文件操作先 `filepath.Clean` 并校验在音乐目录/插件目录内
拖库	密码 bcrypt 哈希存储；会话密钥随机；API Key 不在前端明文回显
请求放大	`limitBody` 限制请求体大小；爬取外部 API 均带超时与限流

8.4 标签读写（internal/taglibx）

- 封装 go-taglib（TagLib 的 WASM 移植，无 CGo）。
- 标准化标签键：`title / artist / album / albumartist / genre / year / track / lyrics / comment` + 扩展键（composer/lyricist/type/language/描述/光盘/总盘/总轨/增益/峰值/音频属性）。
- 支持多艺术家（`/` 与 `_` 拆分）、多值读取、封面（APIC）与歌词（USLT）写入。
- 写操作统一走 `refreshTrack`：更新内存 + 落库（并保持列表按扫描顺序）。

8.5 扫描（main.go /api/scan）

- 异步增量扫描：按目录遍历音频扩展名，边扫边推（`/api/scan/status` 轮询），不等待全部完成。
- 每个文件：读标签 → 探测格式（时长/码率/采样率）→ 计算 SHA256 → 检测乱码/繁体/待识别标记 → 落库。
- 文件名解析（当标签缺失时）：`艺术家 - 标题.后缀` 拆分；多后缀先按音频格式探测。
- 暂停/继续：`POST /api/scan/pause`。排序：扫描中按扫描顺序（seq），完成后可点表头排序。

8.6 刮削引擎（internal/scrape）

- `MultiSource`：按源注册表调度，`auto` 自动按语言优选（中文 → 国内源，外文 → 国际源）；也可手动指定源。
- 源清单（全部插件化）：网易云 / QQ / 酷狗 / 酷我 / 咪咕 / B站 / 汽水 / 波点 / 5sing / MusicBrainz / iTunes / ListenBrainz / Deezer / Last.fm / Spotify / Discogs / Jamendo。
- 每个源实现统一 `Source` 接口：`Search(query) → SearchLyrics(title, artist)`。
- 批量任务 `/api/scrape/batch`：后台执行，进度 + 成功/失败/跳过统计，刷新/换设备可恢复。

8.7 批量任务引擎（scrapejob.go）

统一任务模型（批量刮削 / 一键刮削 / 格式转换 / 乱码修复 / 简繁转换 / 音频识别 / 修正后缀共用）：

- 内存任务表（`jobs`），`GET /api/jobs/{id}` 查询进度（status/done/total/current/results/ok/fail/skip/stage/stageLabel）；`GET /api/jobs` 列出进行中任务。
- 前端启动任务后存 `jobId`，登录/刷新后 `resumeJobs()` 重新挂载轮询——**后台任务跨设备持续执行。**
- 结果统计：成功 / 失败 / 跳过 / 总数，任务结束汇总显示（toast + 任务面板）。
- **取消**：`POST /api/jobs/{id}/cancel`（所有批量任务通用）。

8.8 工具链

模块	功能	说明
convert.go	格式转换	调 ffmpeg 转码（插件化定位），输出同目录不删源文件
fixencoding.go	乱码修复	检测 GBK/UTF-8 错乱（mojibake），重写正确编码；与流水线共用 `repairTrackText`
zhscript.go	简繁体转换	OpenCC：默认繁体→简体，可切换方向
identify.go	音频内容识别	见第 5 节完整流水线
renameclean.go	修正文件后缀	探测真实格式后重命名正确后缀（发如雪.mp3.flac → 发如雪.flac）
activity.go	操作日志	记录"何时刮削了多少首/转换了多少首/是否成功/插件启停"等动作，日志页可删除/清空

9. 插件框架（核心扩展点）

9.1 内置插件模型

```
type Plugin struct {
    Name        string            // 唯一标识：scrape.qq / tool.ffmpeg / download.* / protocol.*
    Label       string
    Kind        PluginKind        // scrape | tool | download | protocol
    Enabled     bool
    Builtin     bool              // 内置不可停用（auto）
    Installable bool              // 支持一键安装
    NeedsKey    bool              // 依赖外部 API Key（默认关闭）
    KeyField    string            // 对应插件配置内 Key 键名
    KeyConfigured bool            // 快照时计算
    Fields      []PluginField     // 前端渲染表单
    Config      map[string]string // 始终序列化（至少 {}）
}
```

- `plugin.go`：注册表（`registerPlugin` / `pluginsSnapshot` / `applyPluginState` / `loadPluginsFromDB` / `savePluginsToDB`）。
- `plugins_api.go`：启动时注册全部刮削源 + FFmpeg + AcoustID；`GET /api/plugins`（快照）、`POST /api/plugins`（启停/配置更新）。
- 免 Key 刮削源在 `loadPluginsFromDB` 中**强制开启**；需 Key 源默认关闭，配置 Key 后自动变为可开启；**删除 Key 保存后自动关闭该插件。**
- 持久化：全量写入 `settings.plugins` 键（JSON），并同步兼容旧键。
- **刮削源 Key 配置已移植到插件内**（设置页不再显示 Key 输入与提示）。

9.2 FFmpeg 工具插件

- 镜像默认**不装** ffmpeg；`LNH_INSTALL_FFMPEG=true`（compose）时 `entrypoint.sh` 启动前 `apk add ffmpeg`。
- 插件页：未安装 → 「一键安装」；已安装 → 「卸载」（清空配置 + 停用插件）。
- `GET /api/ffmpeg/status`：探测路径与版本；`POST /api/ffmpeg/install`：容器内安装；`POST /api/ffmpeg/uninstall`：卸载。

9.3 第三方插件（解耦设计）

第三方插件以 **zip 包**导入，宿主与插件完全解耦：宿主只负责「解压、登记、托管面板、转发运行请求」，插件逻辑全部在 zip 内（plugin.json + panel.html + scripts）。

① 导入与声明

```
zip 结构：
  plugin.json     声明（必填）
  panel.html      自带 Web 面板（可选，声明 panel 字段后自动出现 tab）
  scripts/        插件逻辑（可选，宿主按 JSON 协议调用）

plugin.json 示例：
{
  "name": "tool.unlock",
  "label": "音乐解锁",
  "kind": "tool",
  "version": "1.0.0",
  "description": "批量解锁加密音乐",
  "panel": "panel.html",
  "entry": { "type": "http", "port": 8384, "path": "/api/unlock" }
}
```

导入接口：`POST /api/plugins/import`（multipart zip）→ 解压到插件目录 → 校验 `plugin.json` → 注册 → 前端 `plugins-changed` 事件刷新列表；删除接口 `POST /api/plugins/{name}/uninstall`，删除后页面 tab 实时消失。

② Web 面板机制（panel）

- `GET /api/plugins/{name}/panel`：读取插件内 `panel.html` 并以 text/html 返回（防路径穿越：必须位于插件目录内）。

- 前端 `App.vue` 计算属性 `panelPlugins`：有 `panel` 字段的第三方工具插件自动生成子 tab（`panel-{name}`），内容区渲染 `<iframe src="/api/plugins/{name}/panel">` 同源嵌入。
- 任何插件只要在 `plugin.json` 写 `"panel": "xxx.html"` 就自动出 tab，无需改宿主代码。

③ 运行协议（宿主 ↔ 插件）

- `POST /api/plugins/{name}/run`：宿主把请求转发给插件 entry（HTTP 本地端口），返回 JSON；插件内部自行实现扫描/勾选/处理等逻辑。
- `POST /api/plugins/{name}/fs/scan`：插件向宿主请求扫描音乐目录（返回可访问文件列表，供插件面板展示）。
- 插件配置 `GET/POST /api/plugins` 通用（`{name: {enabled?, config?}}`）。

④ 安全边界

- 插件目录严格限定在宿主插件根目录下，zip 解压过滤路径穿越条目；panel 静态服务校验路径前缀。
- iframe 面板默认同源；宿主 CSP 对第三方内容收紧。
- 插件运行接口带鉴权中间件，无法绕过登录。

9.4 新增一个内置插件

```
// 在 plugins_api.go 的初始化函数中加入：
registerPlugin(&Plugin{
    Name:   "download.example",
    Label:  "示例下载器",
    Kind:   PluginDownload,     // 或 PluginProtocol（Subsonic 等）
    Fields: []PluginField{{Key: "url", Label: "fUrl", Type: "text"}},
}, func(name string, enabled bool, cfg map[string]string) {
    // 启停/配置变更时的生效回调
})
```

注册后前端无需改动自动渲染：开关 + 表单 + Key 徽标。下载 / 协议类型当前为预留扩展位（音源、下载器、自动换源、Subsonic 等插件后续接入）。

10. 前端详解（Vue3）

10.1 状态与路由（store.js）

- `state`（reactive）：authed / view（home|app）/ activeTab（list|edit|batch|dedup|panel-*）/ tracks / selected / scanStatus / batchJob / toolJob / version 等。
- 无 vue-router：`activeTab` + `view` 组合控制页面。
- 任务轮询：`resumeJobs()` 在登录/刷新后按 `jobId` 重新挂载进度轮询。
- `checkVersion(force)`：force（手动）→ `/api/version/check` 实时重查；自动 → `/api/version` 读缓存。

10.2 页面结构

视图	说明
`views/Home.vue`	公开首页：品牌 + 介绍 + 搜索区（默认隐藏，未接入）+ 登录入口
`views/Login.vue`	登录弹层（账号默认留空）
`App.vue`	后台框架：顶栏 + 齿轮菜单（主题 / 语言 / 插件管理 / 操作日志 / 系统设置 / 检查更新 / 关于 / 退出）+ 第三方插件动态 tab
`components/MusicTree.vue`	左侧列表：表头冻结；筛选（全部/乱码/繁体/歌词/海报/无艺术家/待识别）；输入即搜歌名；展示全部 29 个字段列
`components/EditPanel.vue`	右侧编辑：载入原标签 + 文件名回填、保存后按"艺术家-标题"重命名、刮削/歌词/封面
`components/BatchPanel.vue`	批量：一键刮削（含**自动去重复选框，默认不勾选**）+ 29 项字段勾选（全选/反选）+ 智能跳过；音频识别 / 格式转换 / 乱码修复 / 简繁转换 / 修正后缀
`components/DedupPanel.vue`	去重：哈希 / 格式 / 标签分组；勾选后批量删除
`components/ExtPanel.vue`	第三方插件 iframe 面板容器（同源加载插件自带 UI）
`views/Settings.vue`	系统设置：改密 + 常规选项（自动修标题/自动重命名/扫描目录/FFmpeg 路径）
`views/Plugin.vue`	插件管理：内置插件组 + 工具插件组 + 第三方插件组（导入 zip）
弹窗组件	`FormatDialog`（转换）、`UpdateModal`（更新提醒）、`AboutModal`（关于卡片）、`Toast`

导航格局：顶栏「列表 / 🏠 / ⚙ 」；齿轮菜单顺序 = **主题、语言、插件管理、操作日志、系统设置、检查更新、关于、退出。**

10.3 国际化、主题与移动端适配

- `i18n.js`：`lang`（zh/en）+ `t(key)` 字典查找；所有用户可见文案走 i18n，新增文案需补中英两处。
- `style.css`：CSS 变量主题（`data-theme="dark|light"`），默认跟随系统。
- **移动端适配**：`@media(max-width:900px)` 顶栏隐藏非关键徽标（保留 newver/batch）、subtabs 横向滚动（scrollbar-width:none）、mtbl 列宽收敛、field-grid 108px 列、输入行 flex 换行；`@media(max-width:600px)` 顶栏紧凑（padding 6/10、brand 13px）、page.merge 全宽、split gap 10、gear-menu 右对齐防溢出、badge.batch 截断 38vw、panel 收紧、tree-scroll 限高。插件页 `.prow` flex-wrap 换行布局。

11. API 参考

> 除标注外均需登录会话。请求头 `X-Requested-With: LNH-MusicTag`（CSRF）。

11.1 会话

方法	路径	说明
POST	`/api/login`	登录（body: user/pass），下发会话 Cookie
POST	`/api/logout`	登出
GET	`/api/me`	当前会话用户（刷新恢复登录态用）
POST	`/api/change-password`	修改密码

11.2 扫描与曲目

方法	路径	说明
POST	`/api/scan`	开始扫描（body: dir）
POST	`/api/scan/pause`	暂停/继续
GET	`/api/scan/status`	扫描进度（running/paused/added/total/done/skipped）
GET	`/api/tracks`	曲目列表（按 seq）
POST	`/api/tracks/delete`	批量删除选中文件（body: {ids} → {failed,removed}）
POST	`/api/tracks/{id}/tags`	写标签（含重命名联动）
GET	`/api/tracks/{id}/cover`	取内嵌封面
POST	`/api/tracks/{id}/cover`	写封面
GET	`/api/tracks/{id}/lrcfile`	外挂歌词文件

11.3 刮削

方法	路径	说明
GET	`/api/scrape/sources`	源列表与开关
GET	`/api/scrape/search`	指定源搜索（query + source）
POST	`/api/tracks/{id}/scrape`	单曲刮削（source + 字段勾选）
POST	`/api/scrape/batch`	批量刮削（ids + source + 字段），返回 jobId
POST	`/api/scrape/onekey`	一键刮削（body: {ids, dedup}；dedup=true 先自动去重），返回 jobId
GET	`/api/lyrics`	歌词搜索/获取

11.4 批量任务

方法	路径	说明
GET	`/api/jobs/{id}`	任务进度（status/done/total/current/results/ok/fail/skip/stage/stageLabel）
GET	`/api/jobs`	进行中任务列表（刷新/换设备恢复）
POST	`/api/jobs/{id}/cancel`	取消任务（置 cancelled）
POST	`/api/convert`	格式转换（target/bitrate/sampleRate/removeSource）
POST	`/api/fix-encoding`	乱码修复（body: {ids}）
POST	`/api/convert-script`	简繁转换（body: {ids, to: simp/trad}）
POST	`/api/identify`	音频内容识别（异步构建指纹库后逐曲目识别）
POST	`/api/rename-clean`	修正文件后缀（探测真实格式重命名）
POST	`/api/set-chorus`	艺术家 ≥3 时改"合唱"

11.5 去重

方法	路径	说明
GET	`/api/duplicates`	重复分组（hash/format/tags；format/tags 组含 keepId 音质最佳）
POST	`/api/duplicates/remove`	删除组内选中文件

11.6 设置 / 插件 / 日志

方法	路径	说明
GET	`/api/settings`	设置（不含 Key，Key 在插件配置内）
POST	`/api/settings`	保存设置
GET	`/api/plugins`	插件快照（含 config 恒非空、KeyConfigured、panel 字段）
POST	`/api/plugins`	更新启停/配置（body: {name: {enabled?, config?}}）
POST	`/api/plugins/import`	导入第三方插件 zip（multipart）
POST	`/api/plugins/{name}/uninstall`	卸载第三方插件
GET	`/api/plugins/{name}/panel`	插件自带 Web 面板（text/html，同源 iframe 嵌入）
POST	`/api/plugins/{name}/run`	插件运行请求（宿主转发给插件 entry）
POST	`/api/plugins/{name}/fs/scan`	插件请求扫描音乐目录

GET	`/api/ffmpeg/status`	FFmpeg 路径与版本
POST	`/api/ffmpeg/install`	容器内一键安装
POST	`/api/ffmpeg/uninstall`	卸载 FFmpeg（清配置+停用插件）
GET	`/api/logs`	操作日志列表
POST	`/api/logs/delete`	删除指定日志
POST	`/api/logs/clear`	清空日志

11.7 更新检查

方法	路径	说明
GET	`/api/version`	缓存快照（current/latest/url/status）
GET	`/api/version/check`	强制实时重查（手动检查用）

响应 `status`：`ok` / `fail`（全部源失败）/ `unknown`（未检查）。前端据此区分"有新版 / 已是最新 / 检查失败"。

12. 部署

12.1 docker compose（推荐）

```
services:
  lnh-musictag:
    image: ghcr.io/lahuolou/lnh-musictag:latest
    container_name: lnh-musictag
    ports:
      - "10248:10248"
    volumes:
      - /mnt/sata3-1/mp3:/music        # 音乐目录（需读写）
      - lnh_config:/config             # 配置卷（密码/Key/数据库）
    environment:
      - LNH_ADMIN_USER=admin           # 仅首次初始化
      - LNH_ADMIN_PASS=                # 留空则随机初始密码打日志
      - ACOUSTID_API_KEY=
      - LNH_INSTALL_FFMPEG=false       # true 时启动自动装 ffmpeg
    restart: unless-stopped
volumes:
  lnh_config:
```

12.2 关键环境变量

变量	作用

`LNH_ADMIN_USER` / `LNH_ADMIN_PASS`	首次启动初始化种子（之后改库）
`LNH_INSTALL_FFMPEG`	true/1/yes/on → entrypoint 启动前安装 ffmpeg
`LNH_CONFIG_DIR`	非容器运行时的配置目录（默认 `/config`）
`ACOUSTID_API_KEY` 等	各 API Key 初始化种子

12.3 构建

```
cd web && npm ci && npm run build   # 前端产物 → web/dist
cd .. && go build -ldflags "-s -w" -o lnh-musictag .
# Docker（CI 已自动）：
docker build -t lnh-musictag .
```

13. 发布流程（CI/CD）

1. **改码**：项目根（唯一真源）修改代码；前端改完 `npm run build`。
2. **验证**：`go vet ./...` + `go test ./...` + `go build`；本地起服务用浏览器冒烟（登录/关键页/console 无错）。
3. **清理**：删除死代码/冗余代码，避免"屎山"；检查安全防护（CSRF/XSS/SQL 注入/路径穿越）。
4. **同步**：robocopy 到发布副本（唯一 `.git`），`git add -A && git commit`（提交信息含 bump 说明）。
5. **推送**：`git push origin main` → GitHub Actions 自动构建并推送 `ghcr.io/lahuolou/lnh-musictag:latest`。
6. **版本**：先递增 `main.go` 的 `AppVersion` 与 `VERSION` 文件再提交（否则更新检查检测不到新版）。
7. **Release**：`gh release create vX.Y.Z --notes-file ...`（tag 必须带 `v`）。
8. **打包**：重新打源码 zip（含 web/dist 与 Dockerfile/compose/entrypoint），校验 dist 完整后交付。

> 用户确认前不发布。修改完先交用户核验，确认后再推送/发 Release/重打 zip。

14. 更新检查机制

- **数据源**（任一成功即用）：① GitHub Releases API（最实时）；② jsDelivr CDN 读 `VERSION` 文件；③ raw.githubusercontent 读 `VERSION` 文件。
- **三态**：`ok` / `fail`（全部源失败）/ `unknown`（未查）。查询失败不再误报"已是最新"。
- **前端**：登录后自动检查一次，有新版弹 `UpdateModal`；手动「检查更新」强制实时重查。
- **版本比较**：语义化数值逐段比较（`parseVersion`），正确处理 `1.4.50 → 1.5.0`、`1.10.0 > 1.9.0`。

15. 常见问题排查

现象	排查方向
更新后界面还是旧的	浏览器/代理缓存旧 index.html：强刷一次；index.html 已 `Cache-Control: no-cache`
检查更新永远"已是最新"	`VERSION` 文件 / Release tag / `AppVersion` 三者未同步递增
刮削/转换任务换设备后看不到进度	前端登录后 `resumeJobs()` 按 jobId 恢复轮询；确认后端 `/api/jobs` 有任务
扫描慢	异步增量扫描默认开启；首次全量计算哈希属正常耗时
乱码筛选不出/误筛	乱码判定基于标签编码检测（GBK/UTF-8 错乱）；正常中文不应被标记；短串保守不判（防误判繁体）
源搜索失败（status 500 / TLS）	源接口上游变动或网络受限；auto 自动跳过失败源
需 Key 源搜不到	插件页显示 🔑 未配置 → 在对应插件内填 Key 并保存（保存后自动开启）
容器内无 ffmpeg 无法转换	compose 设 `LNH_INSTALL_FFMPEG=true` 或插件页「一键安装」或填宿主路径
mp3 转换失败（Unknown encoder libmp3lame）	精简版 ffmpeg 缺 mp3 编码器；换完整版 ffmpeg 或用 flac/wav 等格式
一键刮削误删文件	自动去重默认不勾选；勾选后按"艺术家+标题/同名多格式/SHA256"分组，保留音质最好的一首
批量封面大量失败	多为源限流：重跑一轮即补（封面缓存/重试已内置）；单首仍失败可看结果消息里的"海报失败"原因
第三方插件导入失败 Failed to fetch	zip 包过大或网络问题：确认 zip 含 plugin.json、文件较小、网络可达；导入后需刷新页面加载插件组
插件 tab 删除后仍显示	卸载后刷新页面列表页的 tab 才消失（前端缓存）；已支持删除后实时消失需新版本

16. 测试

- `go test ./...`：核心逻辑单测（版本比较、去重、刮削源、乱码修复、指纹、文件名解析等）。
- 本地 UI 冒烟：起服务（配置目录指向测试库），浏览器验证登录、列表、编辑、批量、插件页渲染与 console 错误。
- 接口验证：curl 登录拿 Cookie 后逐接口检查（含 CSRF 头）。
- 封面链路：小样本（30 首）命中率抽样 → 全量批量 → 失败曲目重跑 → `/api/tracks` 复核 hasCover 落库。

17. 安全说明

- 密码 bcrypt 哈希；会话 Cookie HttpOnly+SameSite。
- 所有 SQL 参数化；文件路径严格校验（含插件面板静态服务防穿越）；标签文本 XSS 净化。
- CSP：`default-src 'self'`；`script-src 'self' 'unsafe-inline'`（第三方插件 panel 内联脚本所需）；`frame-ancestors 'self'`（仅放行同源 iframe 嵌入）。
- 更新检查、刮削、识别等出站请求均带超时与 UA，避免滥用第三方接口。
- 第三方插件：zip 解压过滤路径穿越、panel 目录前缀校验、运行接口带鉴权。
- 建议部署时：容器内仅挂载音乐目录与配置卷；公网访问时置于反向代理后并启用 HTTPS。